```
# Checking the information about the VS
vsc.get_index(vsc_endpoint_name, vs_index_fullname).describe()
```

```
{'name': 'ml_in_action.rag_chatbot.docs_vsc_idx_cont',
 'endpoint_name': 'ml_action_vs',
 'primary_key': 'id',
 'index_type': 'DELTA_SYNC',
 'delta_sync_index_spec': {'source_table': 'ml_in_action.rag_chatbot.pdf_documentation_text',
  'embedding_vector_columns': [{'name': 'embedding',
    'embedding_dimension': 1024}],
  'pipeline_type': 'TRIGGERED',
  'pipeline_id': 'aa31e35b-7af3-4823-ad83-2c803b59cca4'},
 'status': {'detailed_state': 'ONLINE_NO_PENDING_UPDATE',
  'message': 'Index creation succeeded using Delta Live Tables: https://adb-984752964297111.11.azuredatabricks.net#joblist/pipelines/aa31e35b-7af
3-4823-ad83-2c803b59cca4/updates/5a93e1c5-dbfc-4035-a4af-6875819cd5c6',
  'indexed_row_count': 327,
  'triggered_update_status': {'last_processed_commit_version': 1,
   'last_processed_commit_timestamp': '2024-01-03T16:20:14Z'},
  'ready': True,
  'index_url': 'adb-984752964297111.11.azuredatabricks.net/api/2.0/vector-search/endpoints/ml_action_vs/indexes/ml_in_action.rag_chatbot.docs_vsc
_idx_cont'},
 'creator': 'anastasia.prokaieva@databricks.com'}
```

Figure 4.29 – Using get_index with the describe method shows the index configuration

Because we use self-managed embeddings, our VS index is not connected to any model that can convert our input text query into embeddings to be mapped to the one under a database, so we need to convert them first! Here we are again leveraging the BGE embedding model from the Foundational Model Serving API and then passing embedded text to our index for similarity search:

```
import mlflow.deployments
deploy_client = mlflow.deployments.get_deploy_client("databricks")

question = "What are the economic impacts of automation technologies using LLMs?"

response = deploy_client.predict(endpoint="databricks-bge-large-en", inputs={"input": [question]})
embeddings = [e['embedding'] for e in response.data]

results = vsc.get_index(vsc_endpoint_name, vs_index_fullname).similarity_search(
  query_vector=embeddings[0],
  columns=["pdf_name", "content"],
  num_results=3)
docs = results.get('result', {}).get('data_array', [])
pprint(docs)
```

Figure 4.30 – Converting our query text into embeddings and passing to the index for similarity search

Here is the result of the retrieved queries (only the first found one with two columns `pdf_name` and content):